

RETAIL INDUSTRY

Omni-Channel Customer Loyalty & Rewards Management

FINAL PROJECT — CODE + DOCUMENTATION + OUTPUT

This single PDF consolidates the final project overview, CAP data model, service definition, handler logic, UI source, Agile plan, test cases, deployment steps, Build Code prompts and application output preview.

Deliverable	Status
Project Overview	Included
Data Model Design	Included
Service Definition	Included
Handler / Business Logic	Included
Agile Sprint Plan	Included
Test Case Sheet	Included
Deployment Steps	Included
Build Code Prompts	Included
Application Output	Included

1. Project Overview

Objective: Build an omni-channel retail loyalty system that tracks customer purchases, calculates loyalty points, supports redemption and maintains customer tiers.

Flow: Data Model → CAP Service → Handler Logic → UI → Deployment.

2. Data Model Design

Entity	Attributes	Purpose
Customer	ID, name, email, totalPoints, tier	Customer profile and loyalty balance
Transaction	ID, customer, channel, amount, date, points	Purchase and points history
Redemption	ID, customer, pointsUsed, date, remarks	Redemption records
RewardPolicy	ID, policyName, channel, pointsPer100, active	Reward configuration

db/schema.cds

```
namespace loyalty;
using { cuid, managed } from '@sap/cds/common';

entity Customers : cuid, managed {
  name : String(100);
  email : String(150);
  totalPoints : Integer default 0;
  tier : String(20) default 'Bronze';
  transactions: Association to many Transactions
  on transactions.customer = $self;
  redemptions : Association to many Redemptions
  on redemptions.customer = $self;
}

entity Transactions : cuid, managed {
  customer : Association to Customers;
  channel : String(20);
  amount : Decimal(12,2);
  txnDate : DateTime;
  pointsEarned: Integer;
}

entity Redemptions : cuid, managed {
  customer : Association to Customers;
  pointsUsed : Integer;
  redeemDate : DateTime;
  remarks : String(255);
}

entity RewardPolicies : cuid, managed {
  policyName : String(100);
  channel : String(20);
  pointsPer100: Decimal(8,2);
  active : Boolean default true;
}
```

3. Service Definition

srv/loyalty-service.cds

```
@path: '/odata/v4/loyalty'
service LoyaltyService {
  entity Customers as projection on loyalty.Customers;
  entity Transactions as projection on loyalty.Transactions;
  entity Redemptions as projection on loyalty.Redemptions;
  entity RewardPolicies as projection on loyalty.RewardPolicies;

  action recordPurchase(
    customerID : UUID,
    channel : String,
    amount : Decimal(12,2)
  ) returns Transactions;

  action redeemPoints(
    customerID : UUID,
    pointsUsed : Integer,
    remarks : String
  ) returns Redemptions;
}
```

4. Handler / Business Logic

srv/loyalty-service.js

```
const cds = require('@sap/cds');

module.exports = cds.service.impl(function () {
  const { Customers, Transactions, Redemptions } =
    cds.entities('loyalty');

  function tier(points) {
    if (points >= 3000) return 'Platinum';
    if (points >= 1500) return 'Gold';
    if (points >= 500) return 'Silver';
    return 'Bronze';
  }

  this.on('recordPurchase', async req => {
    const { customerID, channel, amount } = req.data;

    if (!customerID || !channel || Number(amount) <= 0)
      return req.error(400, 'Valid customer, channel and amount are required');

    const customer = await SELECT.one.from(Customers)
      .where({ ID: customerID });

    if (!customer) return req.error(404, 'Customer not found');

    const rate = channel.toLowerCase() === 'online' ? 2 : 1;
    const points = Math.floor(Number(amount) / 100 * rate);
    const txID = cds.uts.uuid();

    await INSERT.into(Transactions).entries({
      ID: txID,
      customer_ID: customerID,
      channel,
      amount,
      txnDate: new Date(),
      pointsEarned: points
    });

    const total = Number(customer.totalPoints || 0) + points;

    await UPDATE(Customers).set({
      totalPoints: total,
      tier: tier(total)
    }).where({ ID: customerID });

    return SELECT.one.from(Transactions).where({ ID: txID });
  });

  this.on('redeemPoints', async req => {
    const { customerID, pointsUsed, remarks } = req.data;

    const customer = await SELECT.one.from(Customers)
      .where({ ID: customerID });

    if (!customer) return req.error(404, 'Customer not found');
    if (Number(pointsUsed) <= 0)
      return req.error(400, 'Points must be greater than zero');
    if (Number(customer.totalPoints) < Number(pointsUsed))
      return req.error(400, 'Insufficient loyalty points');

    const remaining = Number(customer.totalPoints) - Number(pointsUsed);
    const id = cds.uts.uuid();

    await INSERT.into(Redemptions).entries({
      ID: id,
      customer_ID: customerID,
      pointsUsed,
      redeemDate: new Date(),
      remarks: remarks || ''
    });

    await UPDATE(Customers).set({
      totalPoints: remaining,
      tier: tier(remaining)
    }).where({ ID: customerID });

    return SELECT.one.from(Redemptions).where({ ID: id });
  });
});
```

5. Application Source

package.json

```
{
  "name": "retail-loyalty-rewards-capstone",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "cds-serve",
    "watch": "cds watch",
    "build": "cds build"
  },
  "dependencies": {
    "@sap/cds": "^9",
    "@sap/cds-dk": "^9"
  }
}
```

app/loyalty/webapp/index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Retail Loyalty & Rewards</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header><strong>Retail Loyalty & Rewards</strong><nav>
Dashboard | Customers | Transactions | Redemptions | Reward Policy
</nav></header>
<main>
<section class="cards">
<div class="card"><span>Total Customers</span><b>4</b></div>
<div class="card"><span>Total Points Issued</span><b>4,800</b></div>
<div class="card"><span>Total Points Redeemed</span><b>1,450</b></div>
<div class="card"><span>Available Points</span><b>3,350</b></div>
</section>
<section class="panel"><h2>Top Customers</h2>
<table><tr><th>Customer</th><th>Email</th><th>Total Points</th><th>Tier</th></tr>
<tr><td>Aarav Sharma</td><td>aarav.sharma@email.com</td><td>1,250</td><td>Silver</td></tr>
<tr><td>Ananya Reddy</td><td>ananya.reddy@email.com</td><td>2,450</td><td>Gold</td></tr>
<tr><td>Imran Khan</td><td>imran.khan@email.com</td><td>320</td><td>Bronze</td></tr>
<tr><td>Priya Nair</td><td>priya.nair@email.com</td><td>780</td><td>Silver</td></tr>
</table></section>
<section class="panel"><h2>Business Rules</h2>
<ul><li>Online: 2 points per ■100.</li>
<li>Store: 1 point per ■100.</li>
<li>Redemption is rejected if balance is insufficient.</li>
<li>Tier is recalculated after every transaction.</li></ul>
</section>
</main></body></html>
```

app/loyalty/webapp/style.css

```
body{margin:0;font-family:Arial;background:#f5f7fa;color:#1f2937}
header{background:#09213f;color:white;padding:18px 24px;display:flex;justify-content:space-between}
main{padding:24px}.cards{display:grid;grid-template-columns:repeat(4,1fr);gap:16px}
.card,.panel{background:white;border-radius:8px;padding:18px;box-shadow:0 1px 4px #ccd}
.card span{display:block;color:#64748b;font-size:13px}.card b{display:block;font-size:28px;margin-top:8px}
.panel{margin-top:20px}table{width:100%;border-collapse:collapse}
th,td{padding:12px;border-bottom:1px solid #e5e7eb;text-align:left}th{background:#f8fafc}
```

6. Agile Sprint Plan

Sprint	Deliverable
Sprint 1	Entity model, mock data, service definitions
Sprint 2	Point computation and redemption logic
Sprint 3	Fiori dashboard for customer/admin
Sprint 4	Deployment and system testing

Acceptance criteria: CRUD operations work; online/store points are calculated automatically; redemption cannot make points negative; customer tier is updated after points change.

7. Test Case Sheet

ID	Scenario	Input	Expected Result
TC-01	Create customer	Valid customer data	Customer created with Bronze tier
TC-02	Online purchase	■1,000 Online	20 points earned
TC-03	Store purchase	■1,000 Store	10 points earned
TC-04	Redeem points	Redeem 200	200 points deducted
TC-05	Over-redemption	Redeem above balance	Rejected with insufficient-points error
TC-06	Tier update	Cross 500/1500/3000	Silver/Gold/Platinum applied

8. Deployment / Execution Steps

1. Open the project in SAP Business Application Studio.
2. Open Terminal in the project root.
3. Run npm install.
4. Run npm run watch (or cds watch).
5. Open the generated application/service URL.
6. Verify customer, transaction and redemption operations.
7. For BTP deployment, configure the Cloud Foundry target and deploy the application.

Commands

```
npm install  
npm run watch  
npm run build
```

9. SAP Build Code Prompts

Prompt 1 — Create a SAP CAP project for an omni-channel retail customer loyalty and rewards system with Customer, Transaction, Redemption and RewardPolicy entities.

Prompt 2 — Expose the entities through OData V4 and add actions for recording purchases and redeeming points.

Prompt 3 — Implement Online = 2 points per ■100 and Store = 1 point per ■100, with insufficient-balance validation.

Prompt 4 — Create a Fiori/UI5 dashboard with KPI cards, customers, transactions and redemption views.

Prompt 5 — Add validation, tier recalculation and user-friendly error messages.

Prompt 6 — Generate test cases for CRUD, points calculation, redemption and tier changes.

10. Application Output

Prepared output view showing the dashboard, customer details, purchase flow, redemption flow, transaction history and redemption history.

Retail Loyalty & Rewards

Dashboard

Admin User

Dashboard

Customers

Transactions

Redemptions

Purchase

Redeem Points

Reward Policy

Reports

Total Customers

4

+1 this month

Total Points Issued

4,800

+680 this month

Total Points Redeemed

1,450

+250 this month

Available Points

3,350

Top Customers

View All

Customer	Email	Total Points	Tier	Actions
Aarav Sharma	aarav.sharma@email.com	1,250	Silver	
Ananya Reddy	ananya.reddy@email.com	2,450	Gold	
Imran Khan	imran.khan@email.com	320	Bronze	
Priya Nair	priya.nair@email.com	780	Silver	

Recent Transactions

12

Last 7 days

Recent Redemptions

5

Last 7 days

Back

Record Purchase

Admin User

Customer *

Aarav Sharma (aarav.sharma@email.com)

Channel *

Online

Amount (₹) *

1250.00

Transaction Date *

12/06/2025

Points Calculation

Channel: Online (2 points per ₹100)

Amount: ₹1,250.00

Points to be earned: 25

Reset

Submit Purchase

Transactions

Admin User

+ New Purchase

Txn ID	Customer	Channel	Amount (₹)	Points Earned	Date & Time	
TXN-1001	Aarav Sharma	Online	1,250.00	25	12 Jun 2025, 10:15 AM	
TXN-1002	Ananya Reddy	Store	750.00	7	11 Jun 2025, 06:45 PM	
TXN-1003	Priya Nair	Online	2,300.00	46	10 Jun 2025, 09:30 AM	
TXN-1004	Imran Khan	Store	450.00	4	09 Jun 2025, 03:20 PM	
TXN-1005	Aarav Sharma	Store	980.00	9	08 Jun 2025, 01:10 PM	

Showing 1 to 5 of 12 entries

1

2

3

Customer Details

Admin User

Dashboard / Customers / Ananya Reddy

AR

Ananya Reddy

ananya.reddy@email.com

Gold Tier

Total Points

2,450

Points Redeemed

950

Available Points

1,500

Overview

Transactions

Redemptions

Profile

Customer Since

15 Jan 2024

Phone

+91 98765 43210

Total Purchases

8

Membership Tier

Gold

Tier Progress

You need 550 more points to reach Platinum

2,450 / 3,000

Tier Benefits (Gold)

9% discount on all purchases

Priority customer support

Exclusive offers and early access

Back

Redeem Points

Admin User

Customer *

Ananya Reddy (ananya.reddy@email.com)

Reward *

Discount Voucher ₹200

Points Required

200

Available Points

1,500

Points Required

200

Remaining After Redemption

1,300

Remarks

Redeeming voucher for June sale

Reset

Redeem

Redemptions

Admin User

+ New Redemption

Redeem ID	Customer	Reward	Points Used	Date & Time	Remarks	
RED-1001	Ananya Reddy	Discount Voucher ₹500	500	12 Jun 2025, 11:20 AM	June sale voucher	
RED-1002	Aarav Sharma	Free Shipping	100	11 Jun 2025, 02:15 PM	Free shipping benefit	
RED-1003	Priya Nair	Discount Voucher ₹200	200	10 Jun 2025, 04:50 PM	Weekend offer	
RED-1004	Imran Khan	Discount Voucher ₹100	100	09 Jun 2025, 05:35 PM	First purchase reward	
RED-1005	Ananya Reddy	Premium Gift	1,000	08 Jun 2025, 12:40 PM	Birthday reward	

Showing 1 to 5 of 8 entries

1

2

11. Output Explanation

Dashboard: KPI cards show customer and point totals.

Customer Details: Shows profile, tier and point balance.

Record Purchase: Online/store channel determines points earned.

Redeem Points: Validates the available balance before deduction.

Transactions and Redemptions: Provide an auditable activity history.